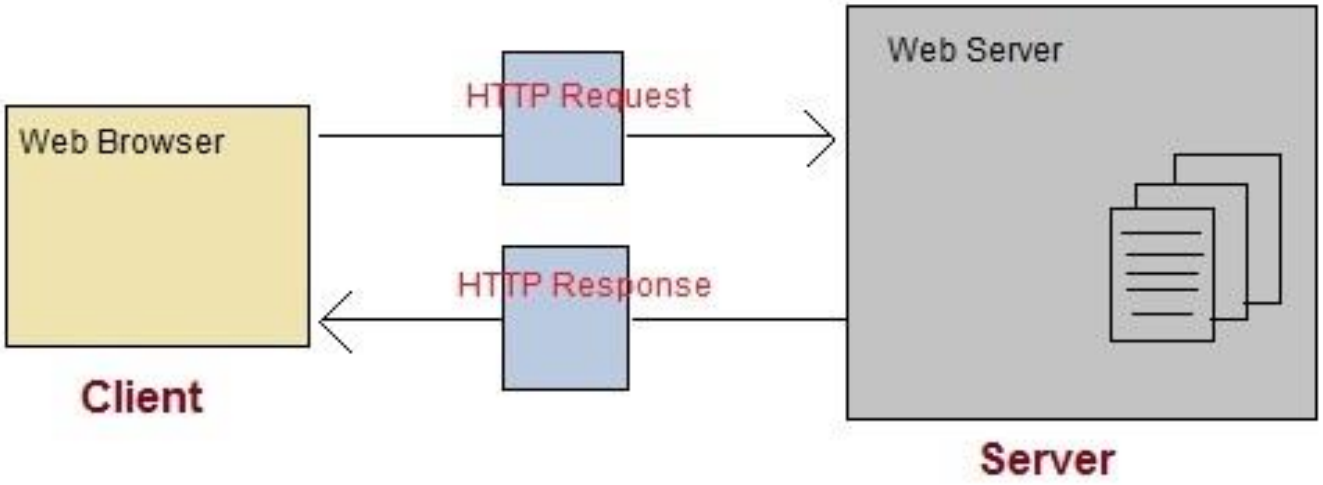
Java Servlets

Introduction



•The client sends an HTTP request and the server answers with an HTML page to the client, using HTTP.

•HTTP is a stateless protocol i.e HTTP supports only one request per connection i.e. with HTTP, the clients connect to the server to send one request and then disconnects.

HTML

- Originally web sites comprised of static HTML documents.

- No way for client to transmit information to the server
  - To customize information to be returned by web server.
  - To store data in the database at server end.

Common Gateway Interface

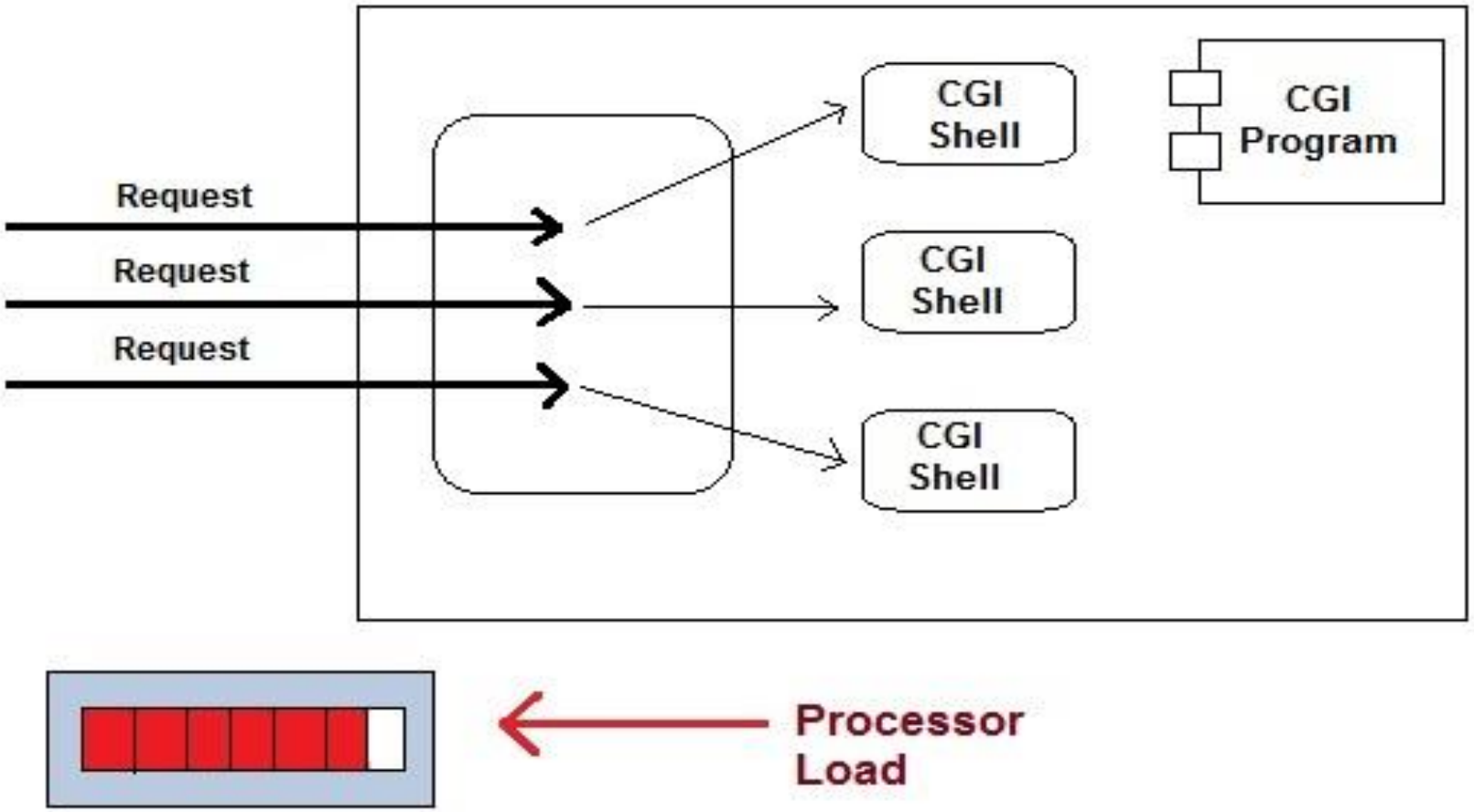
- Evolution of a technology to support e-Commerce and online customer support.

- In combination with HTML forms, CGI provided a mechanism for two-way communication between the client and the server.

Drawbacks of CGI

- CGI can be very slow.
- CGI is hard on server performance.
- Improperly coded CGI can cause security risks.
- CGI has a difficult time in maintaining state (i.e. the program terminates at the end of each request).
- Functionality is limited.
- The amount of data being passed to a CGI routine is limited.

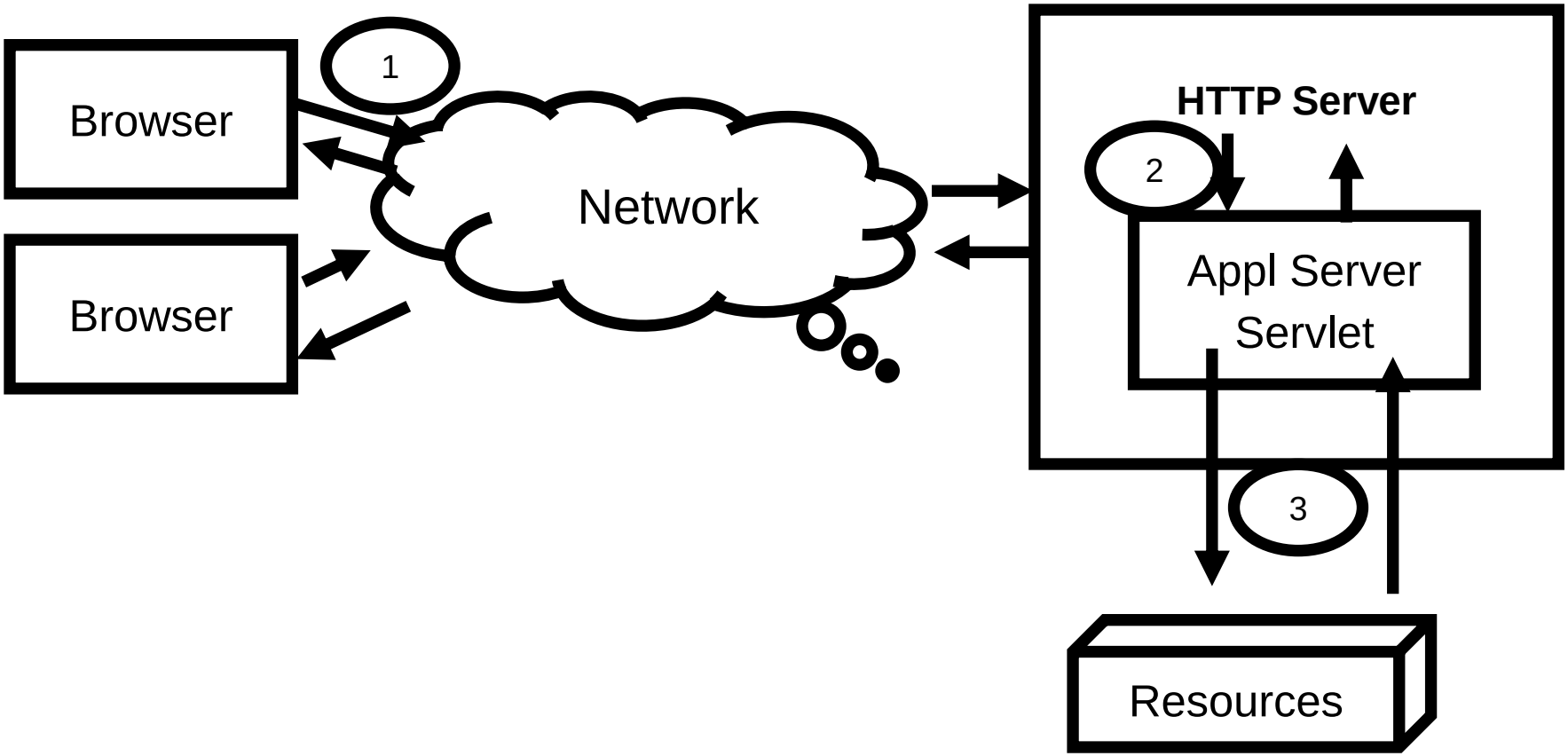
Limitation of CGI



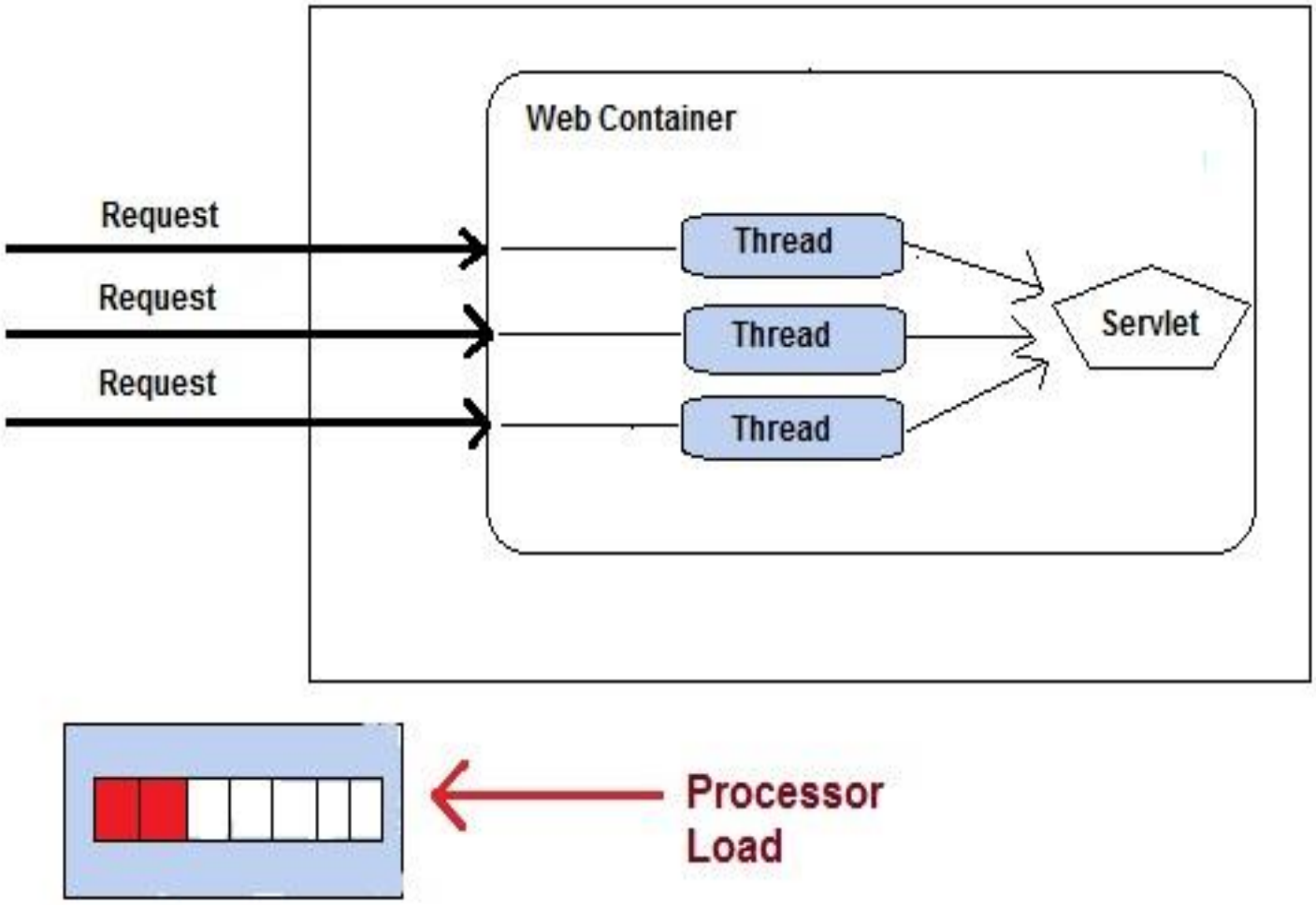
Servlets

- A server-side component written in Java that resides on a web server (such as Apache Tomcat)
  - These servers are also called web containers or servlet containers
- extends the functionality of a web server Dynamically
- works behind the scenes
- build Web pages on the fly when
  - The Web page is based on data submitted by the user.
  - The data changes frequently.
  - The Web page uses information from corporate databases or other such sources.

Servlet Model



Major Benefit of Servlets



Strengths of Servlets

Efficient

- With traditional CGI, a new process is started for each HTTP request.
- With servlets, the Java Virtual Machine stays up, and each request is handled by a lightweight Java thread, not a heavyweight operating system process.

Strengths of Servlets

Convenient as servlets have an extensive infrastructure for

- automatically parsing and decoding HTML form data
- reading and setting HTTP headers
- handling cookies
- tracking sessions, and many other such utilities.

Strengths of Servlets

- Complied
- Crash Resistant
- Cross-Server
- Protocol Independent
- Dynamic Loading

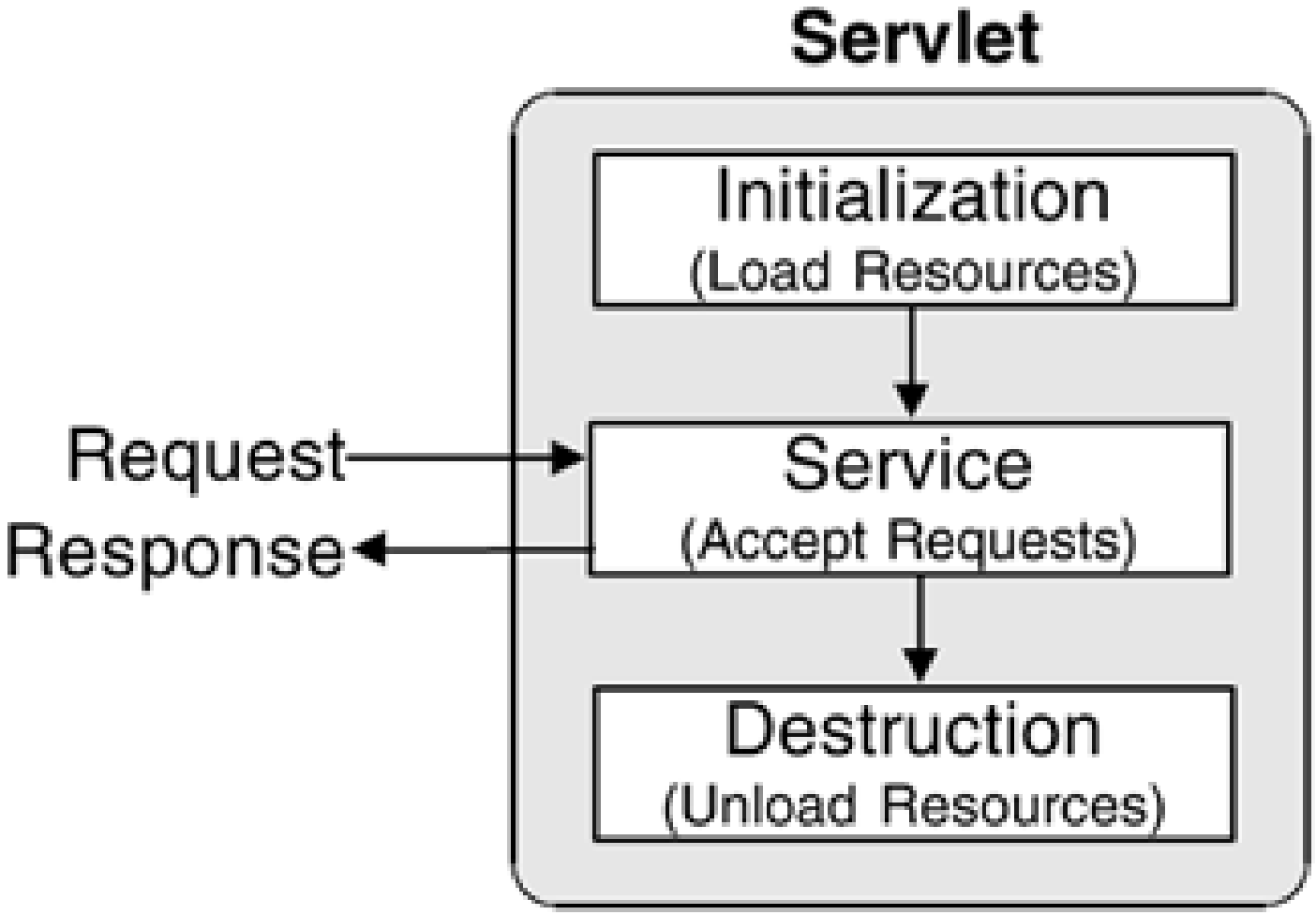
Servlets can

- Dynamically build and return an HTML file based on the client request.
- Process user input passed by an HTML form and return an appropriate response.
- Provide user authentications and other security mechanisms.
- Interact with server resources such as databases, other applications and network files.
- Forward requests from one server to another for load balancing purposes.

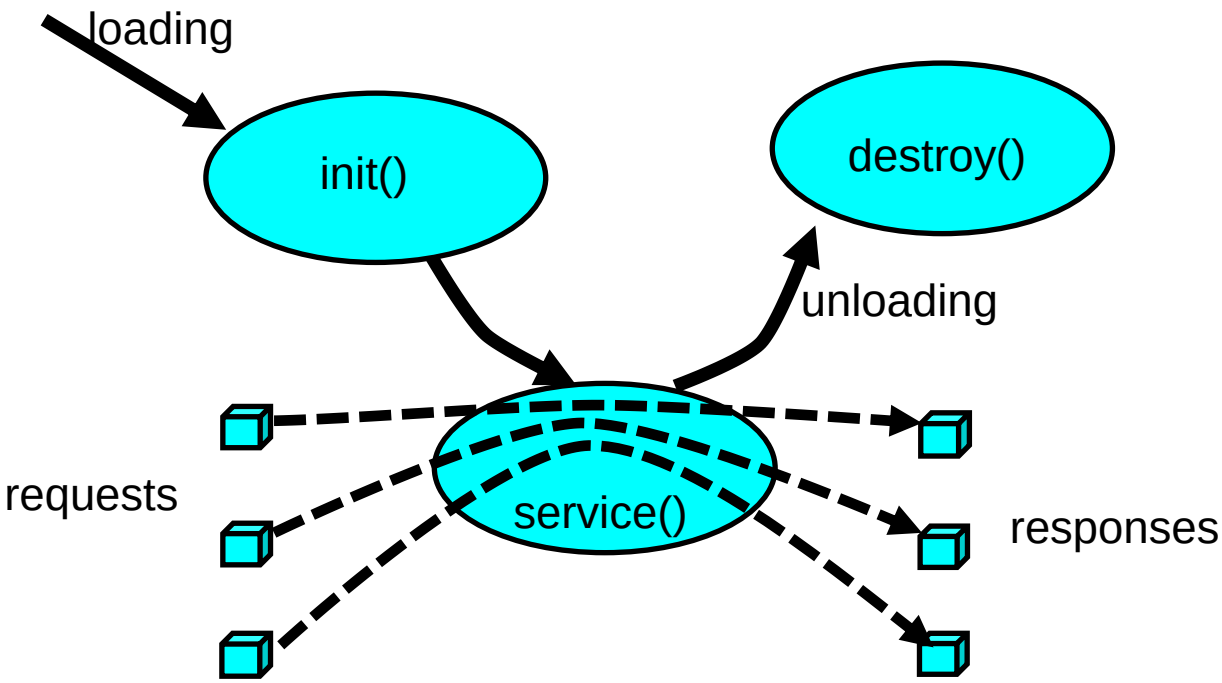
Servlets can

- Facilitate communication between groups of people by publishing information submitted by many clients or multiplayer games.
- Automatically attach Web page design elements, such as headers or footers, to all pages returned by the server.
- Virtually any other way which can enhance or extend a server.

Servlet Life Cycle



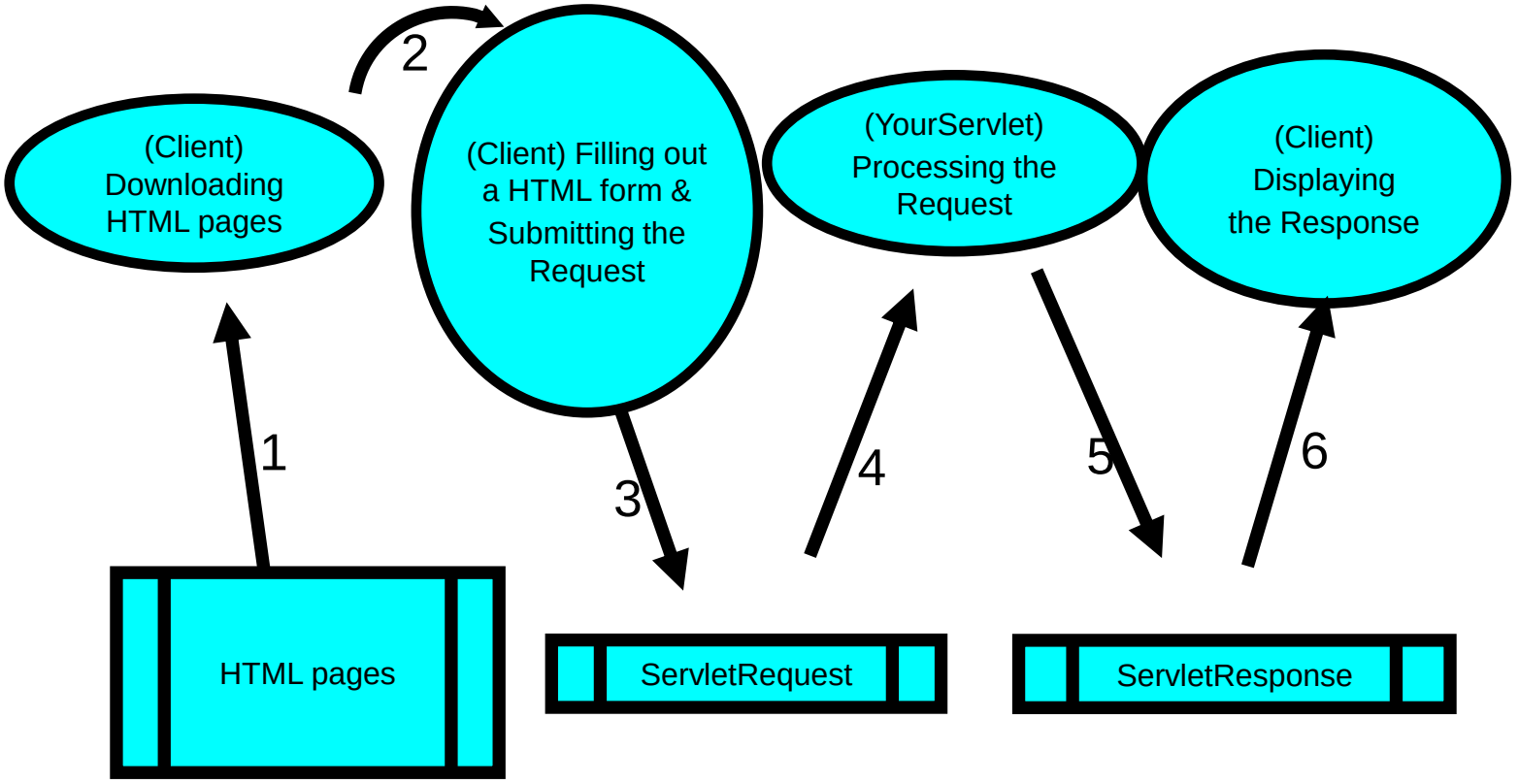
Servlet Lifecycle



Servlet Lifecycle

- The server calls the init method once, when it loads the servlet, and will not call it again unless it is reloading the servlet.
- Initialization is allowed to complete before client requests are handled (that is, before the service method is called) or the servlet is destroyed.
- After the server loads and initializes the servlet, the servlet is able to handle client requests.
- Each client's request has its call to the service method that runs in its own servlet thread: the method receives the client's request, and sends the client its response.

Data Flow


(Client)
Downloading
HTML pages
2
(Client) Filling out
a HTML form &
Submitting the
Request
(YourServlet)
Processing the
Request
(Client)
Displaying
the Response
1
3
4
5
6
HTML pages
ServletRequest
ServletResponse

The basic flow of how servlets work:

•The client user enters the URL which contains the name of an html file in their browser location field. For example:

http://sysname:port/Myhtml.html

•This causes the html tags included in the html file to be interpreted and an html form is rendered by the browser.

•The client user, enters the appropriate information in the form and clicks on a button to submit the request.

•The action of the submit button is coded to execute a servlet

•The servlet is invoked by the Application server, it uses the ServletRequest object which is passed to it as a parameter to determine what it should do.

•After executing any business logic, the servlet writes its response back to the client. The servlet writes a ServletResponse object as output to return the response.

•The Application server uses the the response body of the ServletResponse object to send the response back to the client.

Important Packages

javax.servlet

- Basic servlet definitions, e.g. What is a servlet what are the inputs and outputs ...
- Not tied to any specific protocol (e.g., HTTP)

javax.servlet.http

- HTTP related definitions, extension of the basic interfaces to handle the HTTP protocol

Writing a Servlet

- Very Similar to applet
- Inherits class
  - HttpServlet
- implements one of methods
  - doGet()
  - doPost()
  - doPut()
- Receives objects
  - HttpServletRequest
  - HttpServletResponse

Creating Web Pages on the fly

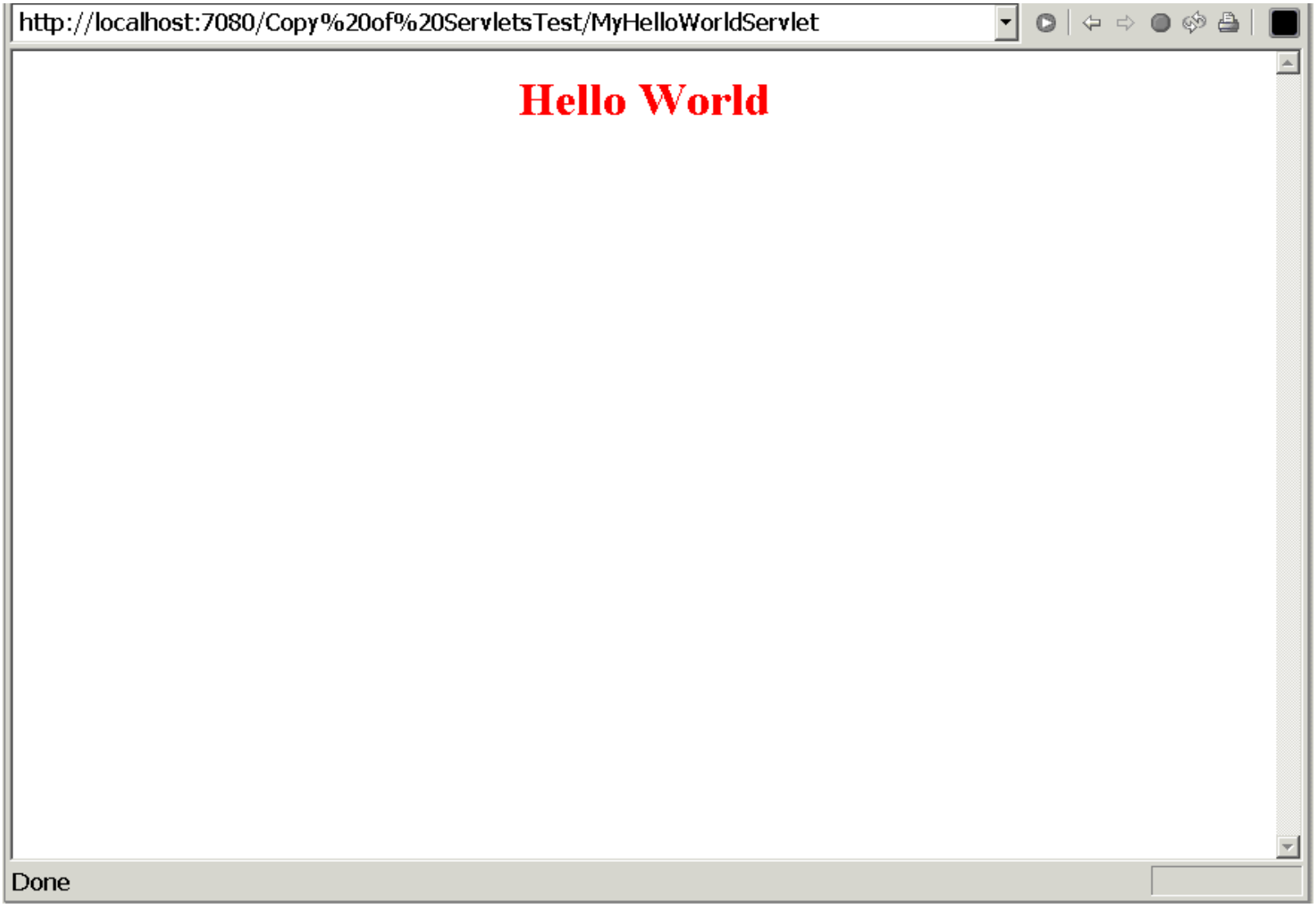
```
import java.io.*;

import javax.servlet.*;
import javax.servlet.http.*;

public class HelloWorldServlet extends HttpServlet {

  public void init() throws ServletException {
  //  Initialize and run when loaded.  Can use default.
  }
  public void destroy() {
  // Release resources, exit, etc. Can use default.
  }
  public void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
  {
                res.setContentType("text/html");

                PrintWriter out = res.getWriter();
                out.println("<html>");
                out.println("<head><title>Hello World</title></head>");
                out.println("<body>");
                out.println("<h1>Hello World</h1>");
                out.println("</body></html>");
                out.close();
  }
}
```


http://localhost:7080/Copy%20of%20ServletsTest/MyHelloWorldServlet
Hello World
Done

There are several way to access a servlet:

- You can explicity enter a URL using the /servlet/ keyword. e.g. to run the HelloWorldServlet, you enter:

  http://server:port/servlet/HelloWorldServlet

- You can embed the servlet access in an action tag in an HTML file. e.g.

  ```
  <form method=GET action=/servlet/HelloWorldServlet>
  </form>
  ```

- Use HTML servlet tags:

  ```
  <servlet name=HelloWorldServlet  code=HelloWorldServlet.class>
  </servlet>
  ```

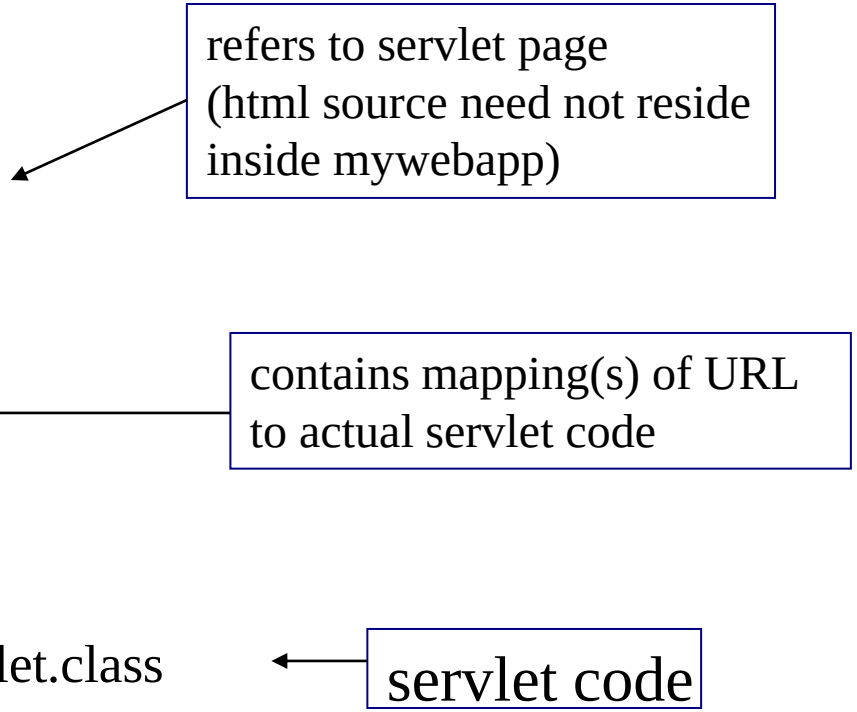
Web application structure

- A web application consists of several files placed inside a context root folder structure: <context-root-folder-name>
  - html file(s) referring to servlet pages
  - WEB-INF
    - web.xml
    - classes
      - <package-name>
        - » Java servlet class file(s)

Web-application example

- SimpleExample:

- mywebapp
  - welcomeform.html
  - WEB-INF
    - web.xml
    - classes
      - servlets
        - » WelcomeServlet.class



web.xml

- web.xml is known as deployment descriptor
- Most important portions:
  - Establishing aliases: associate servlet name to actual servlet code
  - Mapping: associates a URL to a servlet name

web.xml example

```
<web-app>
   <servlet>
      <servlet-name>welcome</servlet-name>
      <servlet-class>
        servlets.WelcomeServlet</servlet-class>
   </servlet>
   <servlet-mapping>
      <servlet-name>welcome</servlet-name>
      <url-pattern>/welcome</url-pattern>
   </servlet-mapping>
</web-app>
```

HTML form example

```
<form action="/mywebapp/welcome" method="get">
    TYPE IN SOME TEXT
    <input type = "text" name ="firstname" />
    <input type = "submit" value="Click" />
</form>
```

Form methods

- Two types of form submission methods
  - GET: form data is appended to the URL (data separated from the action URL by question mark)
    - Use this for query-type actions or idempotent actions
  - POST: form data is “passed” or included as a message to the web-server

HttpServlet class

- Class that servlets need to extend
- Expect to override at least one of the following methods:
  - protected doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException
  - protected doPost(HttpServletRequest request,HttpServletResponse response) throws ServletException, IOException
- Read form data through the **request** parameter, generate output/resulting webpage through the **response** parameter

HttpServletRequest

- Most important method: getParameter()
- Given the input parameter name (indicated in the HTML form), returns a string that represents the associated form data
- May need to use Java conversion features (such as Integer.parseInt()) to convert to the appropriate type for processing
- Examples:
  - String name = request.getParameter("firstname");
  - int year = Integer.parseInt(request.getParameter("year"));

HttpServletResponse

- Used to facilitate result of processing a form
- “generates” html content
- Invoke the getWriter() method to get a PrintWriter handle, then issue print, println methods on that handle
- Invoke getContentType(“text/html”) to specify that you are generating html content
- Example:
  - response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<h2> Welcome </h2>");

Complete doGet() example

```
protected void doGet( HttpServletRequest request, HttpServletResponse
response ) throws ServletException, IOException
{
        String name = request.getParameter( "firstname" );
        int favoriteNumber = name.length();

        response.setContentType( "text/html" );
        PrintWriter out = response.getWriter();

        out.println( "<h2> Welcome " + name + "</h2>" );
        out.println( "Your favorite number is “
                        + favoriteNumber );
        out.close();
}
```

Connecting to a database

- Combine JDBC concepts and servlets
- Better to separate code that connect to the database
  - Tip: have a collection of methods that carry out query or update methods on the database and then invoke these methods from the servlet
- Database code could be in a separate package or be part of the package containing servlet code

doGet() with database access

```
protected void doGet( HttpServletRequest request, HttpServletResponse response )
        throws ServletException, IOException
{
    String name = request.getParameter( "firstname" );
    String mobileNumber;
    try
    {
        mobileNumber = DBAccess.getNum( name );
    }
    catch( Exception e )
    {
        mobileeNumber = "no mobile number";
    }
    response.setContentType( "text/html" );
    PrintWriter out = response.getWriter();

    out.println( "<h2> Welcome " + name + "</h2>" );
    out.println( "Your MOBILE number is " + mobileeNumber );
    out.close();
}
```


getNum()
is a method of
DBAccess.java
and contains
JDBC code

Session Tracking

- Needed when the server wants to keep track of a user (i.e., browser)
  - Hold a shopping cart for the user
  - Save authentication information
- No session mechanism in HTTP servers
- Servlet API provides session API
- Only live for duration of browser session
- Do not persist to disk

References

- https://www.tutorialspoint.com/servlets/index.htm
- https://www.youtube.com/watch?v=lmeEPzsaR-w
- https://www.studytonight.com/servlet/